# Tests para la mini API Tasks: TDD y MockMvc

José MSA

---

## Capítulo 2 bis — Tests para la mini API “Tasks”

**Objetivo:** aprender a probar una API REST en Spring Boot de forma incremental: desde “arranca el contexto” hasta pruebas web con **MockMvc** y una **prueba de integración** con servidor real.

**Stack de pruebas (Maven):** `spring-boot-starter-test` (JUnit 5, AssertJ/Hamcrest, MockMvc, Jackson test). **Comandos:** `./mvnw -q -Dtest='*' test` o simplemente `./mvnw test`.

---

### 0) Punto de partida y pequeño ajuste

Partimos del ejemplo **simplificado** (sin servicios/BD):

- `@RestController` → `/api/tasks`
- Endpoints: `GET /api/tasks`, `GET /api/tasks/{id}`, `POST /api/tasks`

Para poder **probar también validación**, añadimos un DTO mínimo para crear tareas:

```
// src/main/java/com/example/tasks/web/CreateTaskDTO.java
package com.example.tasks.web;

import jakarta.validation.constraints.NotBlank;

public record CreateTaskDTO(@NotBlank String title) {}
```

Y ajustamos el POST del controlador:

```
@PostMapping
public ResponseEntity<Task> create(@RequestBody @jakarta.validation.Valid CreateTaskDTO dto)
    Task saved = new Task(nextId++, dto.title(), false);
    tasks.add(saved);
    return ResponseEntity.created(URI.create("/api/tasks/" + saved.getId())).body(saved);
}
```

**Qué ganamos:** ahora podemos comprobar también el **400 Bad Request** cuando `title` es vacío o falta.

---

### 1) Test 0 — “Arranca el contexto” (sanity check)

**Qué probamos:** que la aplicación **carga el contexto** de Spring Boot sin fallos.

```
// src/test/java/com/example/tasks/ContextLoadTest.java
package com.example.tasks;

import org.junit.jupiter.api.Test;
```

```java
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class ContextLoadTest {
  @Test void contextLoads() { /* si no explota, OK */ }
}
```

**Resultado esperado:** verde [OK]; si algo crítico está mal configurado, aquí fallará temprano.

---

## 2) Tests de capa web con MockMvc (slice test)

**Estrategia:** usar `@WebMvcTest(TaskController.class)` para **levantar solo la capa MVC** y probar los endpoints **sin servidor real**.

```java
// src/test/java/com/example/tasks/web/TaskControllerTest.java
package com.example.tasks.web;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.webmvc.test.autoconfigure.WebMvcTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import static org.hamcrest.Matchers.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(TaskController.class)
class TaskControllerTest {

  @Autowired MockMvc mvc;

  @Test
  void list_shouldReturnEmptyArrayInitially() throws Exception {
    mvc.perform(get("/api/tasks"))
       .andExpect(status().isOk())
       .andExpect(content().contentTypeCompatibleWith(MediaType.APPLICATION_JSON))
       .andExpect(jsonPath("$", hasSize(0))); // [] vacío
  }

  @Test
  void create_shouldReturn201AndLocationAndBody() throws Exception {
    String body = """
      { "title": "Aprender Spring Boot" }
      """;

    mvc.perform(post("/api/tasks")
            .contentType(MediaType.APPLICATION_JSON)
            .content(body))
       .andExpect(status().isCreated())
       .andExpect(header().string("Location", matchesPattern("/api/tasks/\\d+")))
       .andExpect(jsonPath("$.id", greaterThan(0)))
       .andExpect(jsonPath("$.title").value("Aprender Spring Boot"))
       .andExpect(jsonPath("$.done").value(false));
  }
```

```java
  @Test
  void get_shouldReturn404WhenNotFound() throws Exception {
    mvc.perform(get("/api/tasks/9999"))
       .andExpect(status().isNotFound());
  }

  @Test
  void create_shouldReturn400WhenInvalid() throws Exception {
    String invalid = """ { "title": "   " } """; // NotBlank → inválido

    mvc.perform(post("/api/tasks")
            .contentType(MediaType.APPLICATION_JSON)
            .content(invalid))
       .andExpect(status().isBadRequest());
  }
}
```

**Qué estamos probando y qué debe salir**

- **GET lista inicial** → `200 OK`, JSON `[]`.
- **POST válido** → `201 Created`, **cabecera `Location`** con `/api/tasks/{id}`, body con `{ id, title, done:false }`.
- **GET inexistente** → `404 Not Found`.
- **POST inválido** (`title` en blanco) → `400 Bad Request`.

  Nota: `@WebMvcTest` crea una **instancia nueva del controlador por test**, así que la lista interna comienza vacía en cada método (pruebas **independientes**).

---

## 3) Test: comportamiento tras crear (flujo dentro de un mismo test)

Si quieres comprobar que **después de crear** aparece en la lista, hazlo **en el mismo método** (porque cada test reinicia el controlador):

```java
@Test
void flow_createThenList_shouldContainTheNewTask() throws Exception {
  // 1) Crear
  mvc.perform(post("/api/tasks")
          .contentType(MediaType.APPLICATION_JSON)
          .content("""{ "title": "Tarea 1" }"""))
     .andExpect(status().isCreated());

  // 2) Listar y comprobar tamaño 1
  mvc.perform(get("/api/tasks"))
     .andExpect(status().isOk())
     .andExpect(jsonPath("$", hasSize(1)))
     .andExpect(jsonPath("$[0].title").value("Tarea 1"));
}
```

**Qué probamos:** un **mini-flujo** coherente; esperado: verde [OK].

---

## 4) Test de integración con servidor real (opcional en esta fase)

**Objetivo:** arrancar la app **entera** en un puerto aleatorio y llamar a los endpoints "de verdad", usando `TestRestTemplate`.

```java
// src/test/java/com/example/tasks/web/TaskIntegrationTest.java
package com.example.tasks.web;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.boot.test.web.server.LocalServerPort;
import org.springframework.http.*;

import static org.assertj.core.api.Assertions.assertThat;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
class TaskIntegrationTest {

  @LocalServerPort int port;
  @Autowired TestRestTemplate rest;

  String url(String path) { return "http://localhost:" + port + path; }

  @Test
  void createAndGet_shouldWorkEndToEnd() {
    // POST
    var req = new HttpEntity<>(
      """
      { "title": "Integración" }
      """,
      new HttpHeaders() {{ setContentType(MediaType.APPLICATION_JSON); }}
    );
    ResponseEntity<Task> created = rest.postForEntity(url("/api/tasks"), req, Task.class);

    assertThat(created.getStatusCode()).isEqualTo(HttpStatus.CREATED);
    assertThat(created.getHeaders().getLocation()).isNotNull();
    Long id = created.getBody().getId();

    // GET
    ResponseEntity<Task> got = rest.getForEntity(url("/api/tasks/" + id), Task.class);
    assertThat(got.getStatusCode()).isEqualTo(HttpStatus.OK);
    assertThat(got.getBody().getTitle()).isEqualTo("Integración");
  }

  // POJO simple para deserializar (coincide con la respuesta)
  static class Task {
    public Long id; public String title; public boolean done;
  }
}
```

**Qué probamos:** todo el **stack real** (mapeo, serialización, pipeline HTTP). **Esperado:** `201` en creación, `200` al leer por id.

---

## 5) Tests de error y “contratos” de la API (cuando añadamos handler)

Más adelante (Cap. 3) añadiremos un `@ControllerAdvice` para **errores uniformes**. En ese momento, completa tests como:

- **400 de validación** → JSON con `{ error, message, timestamp }`

- **404** → idem, con `error = "NOT_FOUND"` (o el que definas)
- **415** (contenido no soportado) y **405** (método no permitido) → verificar código

Ejemplo de aserciones con JSONPath:

```
.andExpect(jsonPath("$.error").value("VALIDATION_ERROR"))
.andExpect(jsonPath("$.message", containsString("title")))
```

---

## 6) Buenas prácticas de testing (checklist)

- **AAA** (Arrange-Act-Assert): organiza cada test en bloques claros.
- **Nombres expresivos**: `create_shouldReturn201AndLocationAndBody`.
- **Un test = un comportamiento** (flujo completo solo si tiene sentido).
- **Independencia**: no dependas del estado de otro test.
- **Verifica status + headers + body** (no solo el body).
- **Datos mínimos**: lo justo para probar el caso.
- **Velocidad**: usa `@WebMvcTest` para la mayoría; deja `@SpringBootTest` para algunos **E2E**.

---

## 7) Qué cobertura estamos logrando

- [OK] **Controlador**: rutas, parámetros, serialización y **códigos HTTP**.
- [OK] **Validación básica**: `@Valid` + `@NotBlank` (400 esperado).
- [OK] **Contrato REST**: `201 + Location` en creación, `404` cuando no existe.
- Con `@ControllerAdvice`: **formato uniforme** de errores y sus tests.
- Más adelante (cuando haya servicios/repos): tests de servicio y `@DataJpaTest`.

---

## 8) Resumen docente

1. Empezamos con **sanity check** (context loads).
2. Probamos la **capa web** con **MockMvc** (rápido y aislado).
3. Cubrimos **casos felices** y **errores** típicos.
4. Cerramos con una **prueba de integración** para ver el sistema real.
5. En el Cap. 3, añadiremos `@ControllerAdvice` y reforzaremos los tests de error con **payload consistente**.
6. Más adelante, al introducir servicios y repositorios, **slicing** de tests:
   - `@WebMvcTest` (controladores),
   - `@DataJpaTest` (repositorios),
   - `@SpringBootTest` (integración).

---